# FindIt

Lost & Found Platform

Full-Stack Web Development Project Guide

**Technology stack:** React.js · Node.js · Express.js · MongoDB

A practical project blueprint explaining what FindIt does, who uses it, what features it includes, how each feature works in simple words, how the data is stored, and how to build the project in stages.

**Purpose: Learning full-stack web development by building a realistic application from the user interface to APIs, database, authentication, image storage, messaging, and deployment.**

Project reference document · Save this guide for future development

# Contents


- 1. Project overview
- 2. Users and roles
- 3. Website pages and screens
- 4. Features explained in simple words
- 5. Complete user journey
- 6. Technology stack and each tool's job
- 7. Database collections
- 8. Important rules and security considerations
- 9. Build plan: from beginner version to advanced version
- 10. Example API routes
- 11. Suggested folder structure
- 12. Definition of a completed project


This guide describes the intended product. Features will be implemented progressively; not every feature needs to be built at the beginning.

# 1. Project overview

**FindIt** is a web application that helps people find lost belongings and return found items to their rightful owners. It can be used in colleges, offices, libraries, shopping malls, and other public places.

For example, a student loses a wallet in college. The student reports it on FindIt. Another student finds the wallet and creates a found-item post. The owner searches for the item, submits a claim, answers verification questions, and communicates with the finder to arrange its return.

**The goal is to build a complete working application—not just a collection of attractive screens. The frontend, backend, database, and supporting services should work together.**

## Main goals

- Make it easy to report lost and found belongings.
- Help users search for relevant items using keywords and filters.
- Give potential owners a way to request an item and provide proof that it belongs to them.
- Let finders and claimants communicate privately.
- Track claim requests until the item is returned or the request is closed.
- Give administrators tools to moderate posts and respond to abuse.

# 2. Users and roles

| Role | What they can do |
|---|---|
| **Guest** | Browse public item posts and search. They must sign in before creating a post or submitting a claim. |
| **Registered user** | Report lost or found items, manage their own posts, submit claims, view claim status, receive notifications, and chat when permitted. |
| **Finder / claimant** | These are actions a registered user may take. A finder manages claims on an item they posted; a claimant requests an item they believe is theirs. |
| **Administrator** | Review reported content, remove spam, manage abusive accounts, review suspicious activity, and see platform statistics. |

# 3. Website pages and screens

## Home page

- Logo and navigation bar.
- Search bar for lost belongings.
- Buttons to report a lost item or a found item.
- Recently reported items and category shortcuts.
- Clear links to sign in, register, and browse items.

## Browse items page

- A list or grid of lost and found posts.
- Item photos, titles, categories, locations, and dates.

- Keyword search and filters for category, location, date range, and post type.
- A detail page for each item.

### Item details page

- Show the item description and relevant public information.
- Show whether it is a lost report or found report.
- Provide a claim action for eligible found items.
- Avoid showing secret ownership clues, personal phone numbers, or private details publicly.

### Create / edit item page

- A form for item name, category, description, location, date, and photo.
- Separate lost-item and found-item post types.
- Validation messages for missing or invalid information.
- Allow the creator to edit or remove their own post, subject to its status.

### Login and registration pages

- Register with name, email, and password.
- Log in with valid credentials.
- Display useful errors when the information is invalid.
- Protect pages and actions that require a signed-in user.

### User dashboard

- View the user's own posts.
- Edit or delete eligible posts.
- See claim requests made by the user and requests received on their found posts.
- Track pending, accepted, rejected, cancelled, and completed requests.

### Claims page

- Submit a claim and answer verification questions.
- Review incoming requests for an item posted by the finder.
- Accept or reject a request.
- Mark the return process completed when appropriate.

### Chat page

- Show conversations connected to an eligible claim.
- Send and receive messages.
- Load previous message history.
- Restrict access to the people who are part of the conversation.

### Notifications page or menu

- Show new claim requests, claim decisions, and new-message alerts.
- Display unread counts and mark notifications as read.

## Admin dashboard

- Review posts reported by users.
- Remove spam or inappropriate content and manage reports.
- Suspend or restrict abusive accounts when justified.
- View counts such as total users, posts, claims, and completed returns.

# 4. Features explained in simple words

## Feature 1 — Registration and login

Users create an account and sign in before posting items or making claims. The backend validates the details, stores a password hash rather than the original password, and issues an authentication token. The application uses the token to identify the signed-in user and protect restricted actions.

**Example: Prem logs in and reports a lost phone. FindIt records that Prem created the post.**

## Feature 2 — Report a lost item

A user describes something they lost. The form can include item name, category, description, last-seen location, date lost, photo, and optional reward information. React sends the form to Express; the backend validates it, stores the photo through an image service, and saves the item's information and image URL in MongoDB.

**Example: Prem reports a black wallet lost in the college library on 28 October.**

## Feature 3 — Report a found item

A user reports an item they found so its owner can locate it. The form uses similar fields to the lost-item form, but the post type is marked as found. Public details should be enough to help the owner recognise a possible match without revealing every identifying clue.

**Example: Another student finds a wallet in the library and posts it on FindIt.**

## Feature 4 — Search and filters

Users can search by a word and narrow results by category, location, date range, and lost/found type. React sends the selected filters to an Express API, and MongoDB returns matching posts. For a large number of posts, the API should support pagination so it does not return everything at once.

**Example: Search for “wallet” and narrow the results to the college library and a recent date range.**

## Feature 5 — Claim ownership with verification

A person who believes a found item belongs to them submits a claim. The claimant provides information that helps establish ownership. The finder reviews it and accepts or rejects the request. The application should keep private clues out of public item descriptions and should avoid treating a claim as proof of identity by itself.

**Example: The finder asks the claimant to describe a distinctive keychain attached to the wallet.**

## Feature 6 — Claim status tracking

Each claim has a status stored in MongoDB. The application shows the current status in the dashboard and updates it when an authorised person changes it.

**Suggested statuses: Pending (waiting for review), Accepted (claim approved), Rejected (claim declined), Cancelled (request withdrawn), and Completed (the item was returned).**

## Feature 7 — Private chat

The finder and claimant can communicate without publishing personal contact details on the item page. Socket.IO can deliver messages in real time, while MongoDB stores message history. The server must check that a user is a participant before allowing them to read or send messages.

**Example: The finder tells Prem where and when the wallet can be collected.**

## Feature 8 — Notifications

The application creates notifications when an important event occurs. The first version can store notifications in MongoDB and show them in a menu. Real-time delivery can be added later.

**Examples: a new claim arrives, a claim is accepted or rejected, or a new message is received.**

## Feature 9 — Admin moderation

Administrators help keep the platform safe and useful. They can review reports, remove spam, take action on abusive accounts, and view summary statistics. Admin permissions must be checked by the backend on every protected API—not just by hiding admin links in React.

**Example: An administrator reviews a reported post and removes it if it violates the platform's rules.**

## Feature 10 — Image uploads

Users select a photo in React. The server validates the file type and size, and the image is uploaded to Cloudinary or another image-storage service. MongoDB stores the resulting URL, and the frontend displays the image in item cards and details.

**Example: A user uploads a photo of a blue backpack and the image appears with the post.**

# 5. Complete user journey

### 1. Prem loses his wallet

Prem signs in and submits a lost-item report with a photo, location, and date.

### 2. Another student finds it

The student posts a found wallet. The post becomes available in FindIt's search results.

### 3. Prem finds a possible match

He opens the found-item page and submits a claim with details that help establish ownership.

### 4. The finder reviews the claim

The finder checks the answers and accepts or rejects the request.

### 5. They coordinate privately

If accepted, they can use chat to arrange a safe collection.

### 6. The item is returned

After the return, the claim is marked completed and the found item is no longer shown as available.

**Important design decision: lost and found posts are separate records. A claim should refer to a specific found item. Similar names or photos may help users notice a possible match, but the system should not automatically declare two posts a match based only on similarity.**

# 6. Technology stack — what each tool does

| Technology | Job in FindIt |
|---|---|
| React.js | Builds the pages and interactive user interface. |
| CSS or Tailwind CSS | Styles the website and makes layouts responsive on mobile and desktop. |
| React Router | Navigates between pages without reloading the entire website. |
| Node.js | Runs JavaScript on the server. |
| Express.js | Defines API endpoints, middleware, and request handling. |
| MongoDB | Stores users, posts, claims, conversations, messages, and notifications. |
| Mongoose | Defines schemas and models and makes MongoDB queries easier to organise. |
| JWT + bcrypt | JWT helps identify authenticated requests; bcrypt hashes passwords securely. |
| Multer + Cloudinary | Multer processes incoming uploads; Cloudinary or a similar service stores images. |
| Socket.IO | Supports real-time chat and optionally real-time notifications. |
| Git + GitHub | Tracks code changes and stores the repository for collaboration and portfolio use. |
| API testing tool | Tests backend endpoints before and while connecting the React frontend. |

We will not install or learn every tool at once. The plan is to introduce each tool when a feature needs it, so you understand why it is being used.

# 7. Database collections

MongoDB stores information in collections. Collections are similar to tables in a relational database, while each document stores one record. Mongoose will help define the expected fields and relationships.

| Collection | Example information stored |
|---|---|
| users | Name, email, hashed password, role, account status, created date. |
| items | Item name, description, category, type (lost/found), location, date, image URL, creator, item status. |
| claims | Found item, claimant, verification answers, status, review details, timestamps. |
| conversations | Participants and the claim or item the conversation relates to. |
| messages | Conversation, sender, message text, timestamp. |
| notifications | Recipient, notification type, message or related record, read/unread status. |
| reports | Reported post or user, reason, reporter, moderation status, admin action. |

## Start with the minimum database

For the first version, create only the users and items collections. Add claims when the claim workflow is implemented, then add conversations, messages, notifications, and reports as those features are built.

## Key relationships

- An item post is created by a user.
- A claim belongs to a specific found item and is submitted by a claimant.
- A finder can review claims on their own found item.
- A conversation has authorised participants; messages belong to that conversation.
- A notification belongs to the user who should receive it.
- A report identifies the content or account being reviewed.

# 8. Important rules and security considerations

- **Password safety:** never store plain-text passwords. Store a secure password hash.
- **Authentication:** check that a user is signed in before allowing protected actions.
- **Authorisation:** users can edit or delete their own posts; only authorised finders can review claims on their items; only admins can perform admin operations.
- **Private chat:** the backend must verify conversation membership before returning messages or accepting a new message.
- **Ownership clues:** do not publish secret identifying details that a fraudulent claimant could copy.
- **Image uploads:** validate file type and size; do not trust the filename or browser-provided content type alone.
- **Input validation:** validate incoming values on the server even if React validates the form.
- **API errors:** return useful error messages without exposing secrets, stack traces, or internal configuration.

- **Environment variables:** keep database credentials, JWT secrets, and service keys out of the public source code and GitHub repository.
- **Claim rules:** define who can accept, reject, cancel, and complete a claim. Prevent unrelated users from changing another person's claim.
- **Abuse prevention:** consider rate limits, content reporting, and moderation for spam or harassment.

**A claim review can help establish that an item belongs to someone, but it is not a guarantee of legal identity. For valuable items, users should arrange a safe handover and avoid exposing sensitive personal information.**

# 9. Build plan: beginner version to advanced version

## Level 1 — Basic working project

- Create the React project and basic page layout.
- Build the home page, navigation, item cards, and item details.
- Create the Node.js and Express server.
- Connect Express to MongoDB using Mongoose.
- Build CRUD APIs: create, read, update, and delete item posts.
- Connect React to the APIs and display real database data.
- Add keyword search and basic category/type filters.

## Level 2 — Complete lost-and-found workflow

- Add registration and login with password hashing and JWT authentication.
- Protect backend routes and enforce user permissions.
- Add photo uploads using an image-storage service.
- Build claim submission and verification questions.
- Implement claim status tracking and dashboard views.
- Add stored notifications and unread/read status.

## Level 3 — Advanced features

- Add private real-time chat with Socket.IO.
- Build the admin moderation dashboard and report workflow.
- Add pagination, sorting, validation, and consistent error handling.
- Test API permissions, claim transitions, uploads, and chat access.
- Deploy the frontend, backend, and database configuration.
- Write a README with screenshots, features, setup steps, and technologies.

## Recommended order

Start with the user interface and basic item posts. Connect the backend and database next. Add authentication before allowing real users to create or modify data. Then build claims, notifications, chat, moderation, and deployment. This order helps you learn each layer without having to debug everything at once.

# 10. Example API routes

An API route is an address the frontend calls to ask the backend to do something. These are suggested routes for planning; exact names can be refined during development.

| Method | Route | Purpose |
|---|---|---|
| POST | /api/auth/register | Create a user account. |
| POST | /api/auth/login | Authenticate a user. |
| GET | /api/items | List and filter item posts. |
| GET | /api/items/:id | Get one item post. |

| Method | Route | Purpose |
| --- | --- | --- |
| POST | /api/items | Create an item post. |
| PATCH | /api/items/:id | Update an eligible item post. |
| DELETE | /api/items/:id | Delete or remove an eligible post. |
| POST | /api/items/:id/claims | Submit a claim for a found item. |
| GET | /api/claims/my | View the signed-in user's relevant claims. |
| PATCH | /api/claims/:id/status | Update a claim status when authorised. |
| GET | /api/conversations | List conversations for the signed-in user. |
| GET/POST | /api/conversations/:id/messages | Read or send messages after checking membership. |
| GET | /api/notifications | List notifications for the signed-in user. |
| GET/PATCH | /api/admin/reports | Review reports using admin permissions. |

The API must check the current user's identity and permissions. Knowing or guessing an item ID or claim ID must not allow someone to access another user's private data.

# 11. Suggested folder structure

A clean folder structure makes it easier to find code and maintain the project. This is a starting suggestion; we can adjust it as the application grows.

```
FindIt/
  client/                         # React frontend
    src/
      assets/
      components/                 # Reusable UI components
      pages/                      # Home, browse, login, dashboard, etc.
      layouts/
      services/                   # API calls
      hooks/
      context/                    # Shared application state, if needed
      utils/
      App.jsx
      main.jsx
  server/                         # Node.js + Express backend
    src/
      config/                     # Database and service configuration
      controllers/                # Request/business logic
      middleware/                 # Authentication, validation, error handling
      models/                     # Mongoose schemas
      routes/                     # API endpoints
      services/                   # Image, notification, and other services
      sockets/                    # Socket.IO handlers, when added
      utils/
      app.js
      server.js
    .env                          # Local secrets; do not commit
  README.md
  .gitignore
```

## What the backend folders mean

- **Routes:** decide which endpoint handles a request.
- **Controllers:** handle the request and prepare the response.
- **Models:** define the structure of data stored in MongoDB.
- **Middleware:** performs checks such as authentication and validation before a request reaches its controller.
- **Services:** contain reusable logic for tasks such as image storage or notifications.

# 12. When is FindIt complete?

A solid first portfolio version should meet these checks:

- A user can register, log in, and log out.
- A signed-in user can create, view, edit, and delete their own eligible item posts.
- Lost and found posts are clearly distinguished.
- Users can search and filter posts.
- Photos can be uploaded and displayed.
- A claimant can submit a claim and see its status.
- A finder can review claims on their own found item.
- Users can communicate privately when the claim workflow permits it.
- Notifications and admin moderation work at a basic level.
- The backend enforces authentication and permissions.
- The project has been tested and deployed.
- A README explains the project, technologies, features, setup, and limitations.

## Short project description for your portfolio

FindIt is a full-stack lost-and-found web application built with React.js, Node.js, Express.js, and MongoDB. It enables users to report lost and found belongings, search and filter item posts, submit ownership claims, track claim status, and communicate privately. The platform also includes image uploads, authentication, notifications, and an admin moderation dashboard.

## Final recommendation

Build FindIt in small, testable stages. Begin with React pages and basic item CRUD, then connect Express and MongoDB. Introduce authentication before allowing real users to manage data. Add claims, images, chat, notifications, and admin tools only after the foundation works.

End of project guide